Operating
system

MODUL 2
PROCESS

Vieny Evelyn

OUTLINE

- *Cara membuat dan mengakhiri process*
- *Perbedaan foreground & background process*
- *Cara pakai system call*
- *Process hierarchy*
- *zombie process*
- *Race Condition*

APA ITU PROSES?

PROSES ADALAH PROGRAM YANG SEDANG BERJALAN

CONTOHNYA:

- *gedit testWeek2.c*
- *Ketikkan kode di gambar di dalam testWeek2.c*
- *gcc testWeek2.c -o testWeek2*
- *./testWeek2*
- *CTRL C untuk stop*

```
int main(void)
{
    while(1);
}
```

Program yang sedang dijalankan ini yang namanya proses

SETIAP PROSES MEMILIKI:

- *PID (Process ID): Nomor Unik setiap proses yang diberikan OS*
- *PPID (Parent Process ID): PID dari process yang membuat proses*
- *Memory space sendiri: setiap proses tidak saling menggangu*
- *Status (running, sleeping, zombie, dll)*

CONTOH 2:

- *gedit test2Week2.c*
- *Ketikkan kode di gambar di dalam test2Week2.c*
- *gcc test2Week2.c -o test2Week2*
- *./test2Week2*

```c
#include <stdio.h>
#include <unistd.h>
int main(void){

        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        return 0;
}
```

YANG KITA DAPATKAN

- *PID berubah setiap kali program dijalankan karena OS membuat proses baru*
- *PPID tidak berubah, selama kamu menjalankan program dari terminal yang sama. (Shell yang sama)*

**Shell : Program yang menjadi perantara antara user dan sistem operasi.*
**Shell adalah program yang membaca dan menjalankan perintah yang kamu ketik. Beda sama terminal (aplikasi interfacenya)*

APAKAH PID BISA HABIS?

APAKAH PID BISA HABIS?

- *cat /proc/sys/kernel/pid_max → untuk melihat maksimum pid yang tersedia*
- *Apabila semua PID nya habis digunakan, sistem tidak bisa membuat process baru*
- *Karena PID yang sudah selesai akan digunakan ulang (recycle)*

SYSTEM CALL: FORK

SYSTEM CALL

- *System call adalah cara bagi program untuk meminta layanan dari sistem operasi (kernel).*
- *Program tidak bisa langsung mengakses hardware. Jadi program harus “minta izin” ke sistem operasi melalui system call. (Panggil sistemnya)*

FORK()

- *Fork(): System call untuk membuat proses baru.*
- *Parent process adalah proses asli yang memanggil fork()*
- *Sistem akan menduplikasi proses yang sedang berjalan*
- *Child process adalah proses baru hasil dari fork().*

Jadi hasil dari Fork() adalah Child Process. Yang memanggil Fork() adalah parents process. Child Process memiliki PID yang berbeda.

CONTOH 3:

- *gedit myHello.c*
- *Ketikkan kode di gambar di dalam myHello.c*
- *gcc myHello.c -o myHello*
- *./myHello*

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    fork();
    fork();
    fork();
    printf("hello! %d %d \n", getpid(),getppid());
    exit(0);
}
```

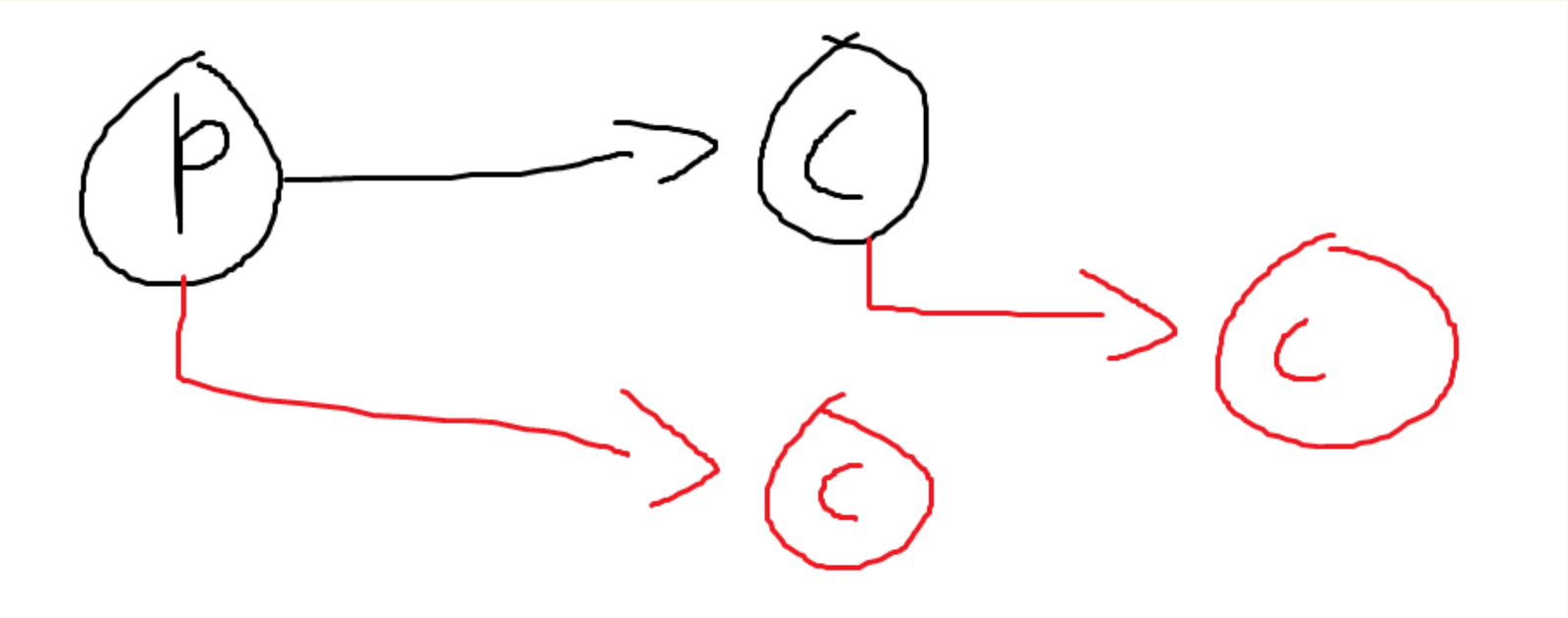
BERAPAKAH HELLO YANG MUNCUL?

YANG KITA DAPATKAN

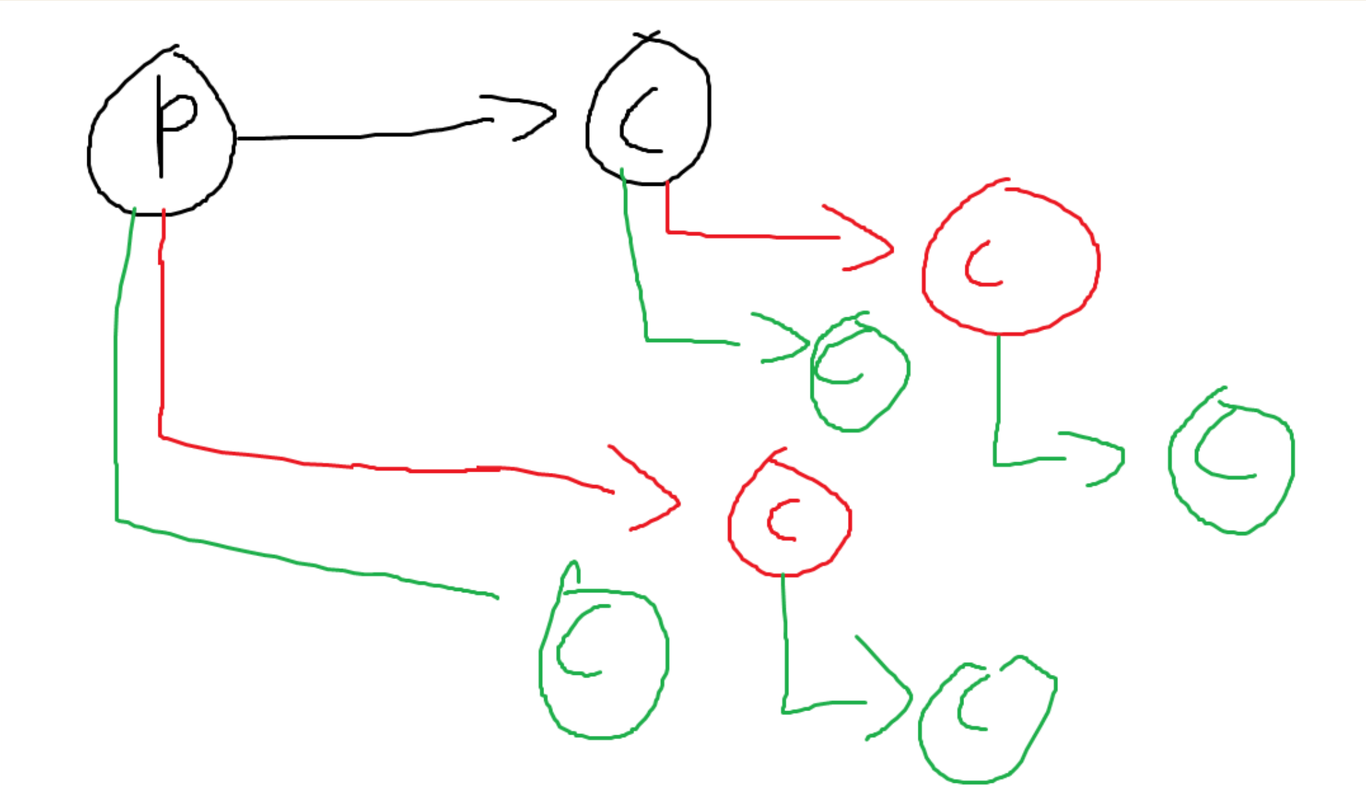
- *Setiap Fork() membentuk sebuah proses*
- *Fork Pertama → akan membentuk 1 proses → Total 2 proses sama parents proses*
- *Fork Kedua → Dari 2 proses, duplikat lagi: Total jadi 4 proses*
- *Fork Ketiga → Dari 4 proses, duplikat jadi 8 proses.*
- *Singkatnya: n fork → 2^n proses*

FORK 1

FORK 2



FORK 3



KENAPA URUTAN PID TIDAK BERURUT?

```
evelyn@LAPTOP-HIRSDE09:~$ ./myHello
hello! PID: 577 PPID:345
hello! PID: 579 PPID:577
hello! PID: 583 PPID:579
hello! PID: 580 PPID:578
hello! PID: 581 PPID:341
hello! PID: 582 PPID:578
hello! PID: 584 PPID:580
hello! PID: 578 PPID:577
```

RACE CONDITION

- *Semua proses tadi berjalan secara bersamaan (pararel)*
- *Oleh karena itu, terjadi race condition*
- *Race condition → balapan. Hasil akhirnya bergantung pada siapa yang selesai duluan tergantung ekseskui programnya gimana urutannya (dari OS Scheduler)*

**Akan dipelajari di week mendatang*

RACE CONDITION

- *Semua proses tadi berjalan secara bersamaan (pararel)*
- *Oleh karena itu, terjadi race condition*
- *Race condition → balapan. Hasil akhirnya bergantung pada siapa yang selesai duluan tergantung ekseskui programnya gimana urutannya (dari OS Scheduler)*

**Akan dipelajari di week mendatang*

APA BEDANYA CHILD SAMA PARENTSNYA?

- *Fork menduplikat semuanya kecuali nilai return value*
- *Parent Process akan mengembalikan nilai PID Child*
- *Child akan mengembalikan PID 0*

- *Program yang tadi tidak mengembalikan return valuenya*

APA BEDANYA CHILD SAMA PARENTSNYA?

- *Fork menduplikat semuanya kecuali nilai return value*
- *Parent Process akan mengembalikan nilai PID Child*
- *Child akan mengembalikan PID 0*

- *Program yang tadi tidak mengembalikan return valuenya*

CHALLENGE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int f1, f2, f3;

    f1 = fork();
    f2 = fork();
    f3 = fork();

    printf("PID:%d | f1:%d f2:%d f3:%d\n",
           getpid(), f1, f2, f3);

    return 0;
}
```

CHALLENGE

```
evelyn@LAPTOP-HIRSDE09:~$ ./myHelloVarian
PID:797 | f1:0 f2:799 f3:800
PID:799 | f1:0 f2:0 f3:801
PID:802 | f1:797 f2:798 f3:0
PID:801 | f1:0 f2:0 f3:0
PID:796 | f1:797 f2:798 f3:802
PID:800 | f1:0 f2:799 f3:0
PID:798 | f1:797 f2:0 f3:803
PID:803 | f1:797 f2:0 f3:0
```

CARI TAHU APA ARTI DARI HASILNYA

CONTOH4:

- *gedit demoFork.c*
- *Ketikkan kode di gambar di dalam demoFork.c*
- *gcc demoFork.c -o demoFork*
- *./demoFork*
- *CTRl + C untuk keluar*

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t parentPID,childPID,x=0;
    printf("Main: x = %d\n",x);
    parentPID = getpid();
    childPID = fork();
    if (childPID == 0)
    {
        printf("In child process\t %d\t%d\n",parentPID,getpid());
        x = 5;
        printf("Child: x = %d\n",x);
        for (;;); /* infinite loop */
    }
    else
    {
        printf("In parent process\t %d\t%d\n",parentPID,childPID);
        x = 11;
        printf("Parent: x = %d\n",x);
        for (;;); /* infinite loop */
    }
}
```

**btw, infinite loop itu cuman supaya prosesnya jalan terus aja buat contoh*

HASIL YANG DIDAPATKAN

- *X awalnya kosong.*
- *Akan tetapi, apabila dia adalah child process maka x akan diganti 5.*
- *Kalau parents, x diganti 11*

```
Main: x = 0
In parent process        parentPID: 549  childPID: 550
Parent: x = 11
In child process         parentPID: 549  childPID: 550
Child: x = 5
```

KALAU CHILDPID DIGANTI KE 3000?

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    pid_t parentPID, childPID;
    int x = 0;

    printf("Main: x = %d\n", x);

    parentPID = getpid();
    childPID = fork();
    childPID = 3000;


    if (childPID == 0)
    {
        // Child process
        printf("In child process\t parentPID: %d\t childPID: %d\n",
               parentPID, getpid());

        x = 5;
        printf("Child: x = %d\n", x);

        for (;;); // infinite loop
    }
    else
    {
        // Parent process
        printf("In parent process\t parentPID: %d\t childPID: %d\n",
               parentPID, childPID);

        x = 11;
        printf("Parent: x = %d\n", x);

        for (;;); // infinite loop
```

BAGIAN INI YANG DIGANTI

HASIL YANG DIDAPATKAN

- *Semua proses childPID nya jadi 3000, jadi child akan masuk ke else karena childpid == 0 nya False.*
- *Apakah Output berasal dari parents aja? Nggak. Output berasal dari child dan parents karena childnya juga jalan*
- *Cuman childpid nya aja yang di timpa 3000*

```
evelyn@LAPTOP-HIRSDE09:~$ ./demoForkVarian
Main: x = 0
In parent process        parentPID: 694  childPID: 3000
Parent: x = 11
In parent process        parentPID: 694  childPID: 3000
Parent: x = 11
```

ZOMBIE:
APA ITU ZOMBIE?

ZOMBIE

- *Setiap Child harus melapor/memberi pesan ke parents apabila sudah menyelesaikan prosesnya.*
- *Zombie terjadi ketika child sudah selesai tetapi tidak memberi laporan kepada parents bahwa ia sudah selesai.*
- *Child selesai mengerjakan tugasnya → tapi parent belum tahu child udh selesai → Child jadi zombie*

**Bisa di cek di entry process table menggunakan command "top" di terimnal lain*

**Contohnya coba dari modul*

ZOMBIE

```
evelyn@LAPTOP-HIRSDE09:~$ ps axj | grep demoFork
    372     799     799     372 pts/0        799 R+    1000   1:08 ./demoFork
    799     800     799     372 pts/0        799 Z+    1000   0:12 [demoFork] <defunct>
    641     804     803     641 pts/3        803 S+    1000   0:00 grep --color=auto demoFork
evelyn@LAPTOP-HIRSDE09:~$
```

*ps axj | grep demofork → melihat proses yang sedang terjadi

<defunct> Z+ = zombie proses artinya

ZOMBIE:
KILLING ZOMBIE

KILLING ZOMBIE: WAITPID()

- *waitpid() → mencegah terjadinya zombie*
- *Parent akan menunggu child selesai dan mengambil status exit nya child*
- *Child proses jalan → Parents menunggu hingga selesai menggunakan waitpid → child selesai → parents menerima pesannya → parents lanjut proses selanjutnya*

NOTES:

- *Apabila Child di kill juga menggunakan command “kill -9”, parents akan tahu apabila menggunakan waitpid() karena parents menunggu child selesai*

**silahkan dicoba dari modul*

THANK YOU

THANKS FOR READING